Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

# Generating source code from UML diagrams

Master's Thesis

2026
bc. Jahonoro Tojieva

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

# Generating source code from UML diagrams

Master's Thesis

| | |
|---|---|
| Study Programme: | Computer Science |
| Field of Study: | Applied Computer Science |
| Department: | Department of Applied Informatics |
| Supervisor: | doc. Ing. Ivan Polášek, PhD |

Bratislava, 2026
bc. Jahonoro Tojieva


Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

# ZADANIE ZÁVEREČNEJ PRÁCE

**Meno a priezvisko študenta:** Bc. Jahonoro Tojieva
**Študijný program:** aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
**Študijný odbor:** informatika
**Typ záverečnej práce:** diplomová
**Jazyk záverečnej práce:** anglický
**Sekundárny jazyk:** slovenský

**Názov:** Generating source code from UML diagrams
*Generovanie zdrojového kódu z UML diagramov*

**Anotácia:** Pri vývoji rozsiahlych softvérových systémov sa štandardne začína vytvorením prípadov použitia z požiadaviek zadávateľa. Jeho dobrému porozumeniu môže pomôcť dynamický model prípadu použitia vo forme BPMN diagramu, UML diagramu aktivít, stavového alebo sekvenčného diagramu.
Ďalším krokom je najčastejšie prvá implementácia prototypu a bolo by zaujímavé, keby tento krok bol automatizovaný klasickým prístupom cez prekladače diagramov do zdrojového kódu a porovnaný s možnosťou generovať kód pomocou AI, ak nemáme pripravené nástroje na štandardný proces transformácie ako súčasť CASE systémov a podobne.

**Cieľ:** Analyzujte možnosti generovania zdrojového kódu z diagramov. Vytvorte prototyp ako plugin alebo nástroj napr. pre draw.io na generovanie zdrojového kódu z UML sekvenčného diagramu. Porovnajte výsledky s možnosťou generovania cez LLM.

**Vedúci:** doc. Ing. Ivan Polášek, PhD.
**Katedra:** FMFI.KAI - Katedra aplikovanej informatiky
**Vedúci katedry:** doc. RNDr. Tatiana Jajcayová, PhD.

**Dátum zadania:** 04.11.2025

**Dátum schválenia:** 11.11.2025 prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

.....
študent

.....
vedúci práce


Comenius University Bratislava
Faculty of Mathematics, Physics and Informatics

# THESIS ASSIGNMENT

| | |
|---|---|
| **Name and Surname:** | Bc. Jahonoro Tojieva |
| **Study programme:** | Applied Computer Science (Single degree study, master II. deg., full time form) |
| **Field of Study:** | Computer Science |
| **Type of Thesis:** | Diploma Thesis |
| **Language of Thesis:** | English |
| **Secondary language:** | Slovak |

**Title:** Generating source code from UML diagrams

**Annotation:** When developing large software systems, it is standard practice to start by creating use cases from the client's requirements. A dynamic model from use case in the form of a BPMN diagram, UML activity diagram, state transition or sequence diagram can help to ensure a good understanding of the requirements. The next step is usually the first implementation of a prototype, and it would be interesting if this step could be automated using a classic approach via diagram-to-source code compilers and compared with the possibility of generating code using AI, if we do not have tools ready for a standard transformation process as part of CASE systems and the like.

**Aim:** Analyze the possibilities of generating source code from diagrams. Create a prototype as a plugin or tool for (e.g.) draw.io to generate source code from a UML sequence diagram. Compare the results with the possibility of generating via LLM.

| | |
|---|---|
| **Supervisor:** | doc. Ing. Ivan Polášek, PhD. |
| **Department:** | FMFI.KAI - Department of Applied Informatics |
| **Head of department:** | doc. RNDr. Tatiana Jajcayová, PhD. |
| **Assigned:** | 04.11.2025 |

**Approved:** 11.11.2025 prof. RNDr. Roman Ďurikovič, PhD.
Guarantor of Study Programme

.....
Student

.....
Supervisor

**Acknowledgments:** Here you can thank your supervisor and/or other persons who somehow helped you with this thesis. You should acknowledge grants/projects in case your research was financed by any.

# Abstract

English abstract: 100-500 words; keep it formatted in one paragraph. Do not use abbreviations and other niche advanced terms here, which will be explained in the thesis later.

**Keywords:** first, second, third, (fourth, fifth)

# Abstrakt

The Slovak abstract: should be the translation of the English one.

**Kľúčové slová:** jedno, druhé, tretie (prípadne štvrté, piate)

# Contents

# List of Figures

# List of Tables

# Introduction

See file `uvod.tex` for information about recommended contents of the Introduction chapter.

# Chapter 1

# Related Work

## 1.1 Model-Driven Development

Model-Driven Development (MDD) treats models as primary artifacts of the software development process and focuses on automatic transformations between different abstraction levels of a software system Sánchez et al. (2010). The objective of this approach is to increase the abstraction level of software engineering activities while reducing repetitive and error-prone manual implementation tasks. MDD enables systematic transitions from requirements models to architectural and implementation models through formally defined transformations.

An important challenge in software engineering is the representation of non-functional requirements such as security, integrity, response time, and authenticity. These concerns typically affect multiple parts of a software system simultaneously and therefore represent crosscutting concerns Sánchez et al. (2010). Traditional object-oriented approaches often lead to scattered implementations of such functionality across multiple modules, which negatively impacts maintainability and system evolution. Aspect-Oriented Software Development (AOSD) addresses this issue by encapsulating crosscutting concerns into separate aspect modules and defining mechanisms for their integration with the base system functionality Sánchez et al. (2010).

The analyzed approach combines MDD with aspect-oriented modeling techniques in order to automate the transformation of aspect-oriented requirements into software architecture models Sánchez et al. (2010). Reusable transformation patterns are applied to recurring non-functional requirements, enabling automatic generation of architectural elements associated with these concerns. The transformation process improves consistency between requirements and architecture while reducing development effort and simplifying maintenance activities.

The proposed methodology further utilizes UML-based scenario models and QVT transformation rules for generating aspect-oriented software architectures Sánchez et al.

(2010). Both structural and behavioral architectural views are generated automatically while preserving aspectual requirements specified during requirements engineering. This demonstrates how model-driven techniques can support automated software architecture generation and improve traceability between requirements and generated system artifacts.

## 1.2 EBNF-Based Modeling Language Definitions

Traditional UML and graphical modeling languages are commonly defined using metamodeling techniques. However, such approaches often require additional textual constraint languages because graphical metamodels alone are insufficient for describing complete syntax specifications Xia and Glinz (2003). An alternative approach based on Extended Backus–Naur Form (EBNF) was proposed for rigorous specification of graphical modeling languages through transformation of graphical elements into equivalent textual representations.

The proposed method maps graphical constructs into textual language elements and formally defines their syntax using EBNF grammar rules Xia and Glinz (2003). This approach separates context-free syntax, context-sensitive syntax, and semantic constraints while simplifying parser construction and improving readability of the language specification. The work demonstrates that grammar-based approaches can also be effectively applied to graphical modeling languages such as UML.

## 1.3 Parser Generation Tools and ANTLR

Parser generators are widely used in software engineering for compiler construction, machine-level translation, natural language processing, and processing structured data formats Ortin et al. (2022). These tools automate the generation of lexical and syntactic analyzers from formally specified grammars, reducing the amount of manually implemented parsing logic. Among the most widely adopted parser generation approaches are the traditional Lex/Yacc toolchain and the modern ANTLR framework Ortin et al. (2022).

ANTLR differs from traditional Lex/Yacc systems through its support for adaptive LL(∗) parsing, automatic parse tree generation, integrated lexer and parser specifications, and support for multiple target programming languages Ortin et al. (2022). In contrast to Yacc-based bottom-up parsing approaches, ANTLR employs a top-down parsing strategy and extended BNF grammar notation, allowing grammars to be expressed more concisely and with improved readability Ortin et al. (2022). The framework also provides additional development support through IDE plugins and vi-

sualization tools such as TestRig, which simplify debugging and grammar validation during parser development Ortin et al. (2022).

An empirical study comparing Lex/Yacc and ANTLR demonstrated that ANTLR significantly improved parser development productivity and maintainability in the context of compiler construction education Ortin et al. (2022). Students using ANTLR completed parser-related tasks faster, required fewer additional development hours, and achieved higher completion rates for compiler assignments compared to students using Lex/Yacc Ortin et al. (2022). The study also reported higher perceived simplicity, intuitiveness, and maintainability for ANTLR-based implementations Ortin et al. (2022). These results suggest that ANTLR provides practical advantages for rapid language development and parser-based software engineering tasks.

ANTLR has also been adopted in real-world software systems and educational environments due to its flexibility and tooling ecosystem. The framework supports automatic parse tree construction, semantic predicates, grammar modularity, and code generation for multiple target languages including Java, Python, C++, C#, and Go Ortin et al. (2022). Such characteristics make ANTLR suitable not only for compiler construction, but also for domain-specific language development and model-driven software engineering applications.

## 1.4 Large Language Models and AI-Assisted Code Generation

Recent advances in Large Language Models (LLMs) have significantly influenced automatic code generation and software engineering research. Models such as ChatGPT, GPT-4, and Llama 2 demonstrate strong capabilities in natural language understanding, program synthesis, and code completion Zhao et al. (2024). These systems are based on transformer architectures and are capable of generating executable source code directly from natural language descriptions. Researchers have increasingly explored the integration of LLMs into software engineering tasks including requirement analysis, modeling, coding, testing, and verification.

Zhao et al. Zhao et al. (2024) proposed an LLM-assisted software development approach called *Xd-CodeGen*, designed for generating large-scale Java applications using ChatGPT 3.5. Unlike earlier studies focused mainly on benchmark datasets such as HumanEval or MBPP, the proposed approach targets practical software engineering workflows for complex systems. The framework divides software generation into four phases: requirement analysis, modeling, code generation, and verification. During requirement analysis, ChatGPT is used to decompose requirements into structured sub-requirements and construct a domain knowledge graph. UML class diagrams and

activity diagrams are then generated during the modeling phase, providing an intermediate representation between natural language requirements and executable code.

The study emphasizes the importance of prompt engineering and iterative prompting when interacting with LLMs. Prompt engineering is described as the process of designing structured prompts containing instructions, context, input data, and expected output formats to improve generation quality Zhao et al. (2024). Iterative prompting is further used to refine generated artifacts by repeatedly providing feedback and correction prompts to the model. The authors report that this iterative interaction significantly improves the correctness and usability of generated UML diagrams, SQL scripts, Java classes, and business logic methods.

Another important contribution of the approach is the integration of knowledge graphs and UML-based model-driven development techniques. The knowledge graph stores domain entities and relationships using Neo4j and Cypher scripts, enabling ChatGPT to access contextual business knowledge during subsequent development stages. UML class diagrams and activity diagrams are represented using PlantUML scripts, allowing automatic transformation between models and code generation tasks. This combination of knowledge graphs, UML modeling, and LLM-based code synthesis improves software traceability and reduces ambiguity in natural language requirements.

The generated software artifacts are further validated using runtime verification at code level. In this phase, Java programs are translated into MSVL representations and verified against formal specifications expressed using Propositional Projection Temporal Logic (PPTL). The verification process attempts to ensure that generated programs satisfy specified temporal properties and functional requirements Zhao et al. (2024). The authors demonstrated the feasibility of the approach through the implementation of a practical Java web-based sales management system containing approximately 20,000 lines of code.

## 1.5 UML-Based Code Generation

Automatic source code generation from UML models represents an important direction of model-driven software engineering. UML diagrams provide abstract representations of software structure and behavior that can be transformed into executable implementations Salunke et al. (2024). Such approaches aim to reduce manual coding effort, improve development efficiency, and simplify rapid software prototyping.

Recent research also investigates integration of Artificial Intelligence, Natural Language Processing (NLP), and Large Language Models (LLMs) into UML-based software development workflows Salunke et al. (2024). These systems attempt to combine UML models, requirement analysis, and automated generation techniques in order to

support intelligent software engineering processes. The combination of UML modeling and AI-assisted generation demonstrates the growing importance of hybrid approaches that integrate formal modeling techniques with modern machine learning methods.

# Chapter 2

# Our Approach

The proposed approach focuses on automated source code generation from UML diagrams using a hybrid parsing architecture. The system combines a custom procedural parser for UML class diagrams with a rule-based language parser for UML sequence diagrams. The overall workflow is designed as a transformation pipeline that converts diagram representations into structured intermediate models and subsequently into source code artifacts.

For UML class diagrams, a custom procedural parser was implemented to process the XML structure exported from draw.io diagrams. The parser extracts classes, attributes, methods, relationships, inheritance hierarchies, and multiplicities directly from the diagram representation. This approach provides full control over the extraction process and enables flexible handling of diagram-specific constructs and variations. The extracted information is transformed into an internal object model that can later be used for source code generation.

For UML sequence diagrams, the approach adopts a rule-based parsing strategy using ANTLR. Instead of directly processing raw XML, sequence diagrams are first transformed into a simplified intermediate textual representation called `.sqd`. This intermediate language describes participants, method calls, return values, and self-calls in a structured and parser-friendly format. ANTLR grammar rules are then used to perform lexical and syntactic analysis of the `.sqd` language, producing an Abstract Syntax Tree (AST). A visitor-based semantic analysis phase extracts interactions between system components and converts them into an internal representation suitable for further processing and code generation.

The proposed architecture separates diagram extraction from language parsing. The extraction layer is responsible for transforming graphical UML diagrams into normalized textual representations, while the parsing layer validates syntax and builds semantic models from these representations. This separation improves modularity, maintainability, and extensibility of the system.

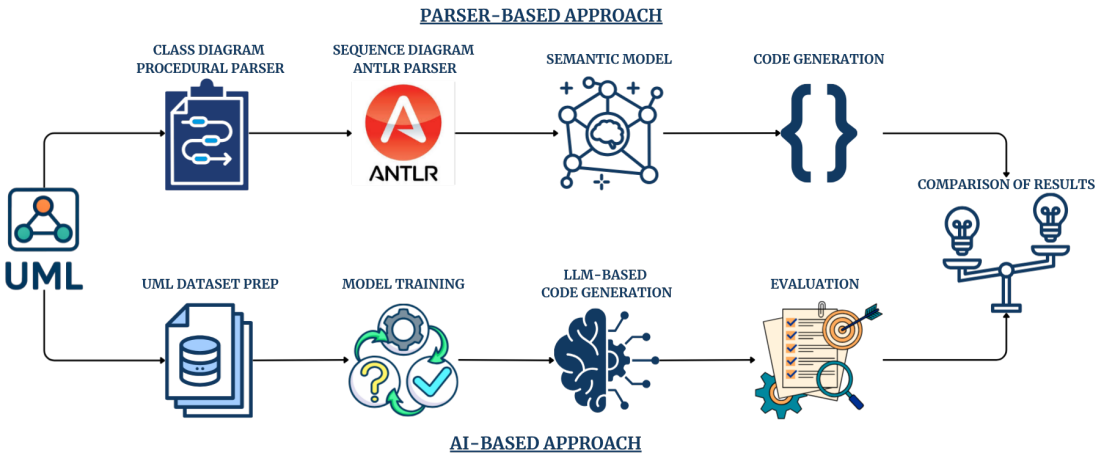


Figure 2.1: System architecture of our approach

Figure 2.1 illustrates the overall architecture of the proposed system and the interaction between parser-based and AI-assisted components.

The implementation pipeline consists of several phases:

1. Extraction of UML diagrams from draw.io XML files.
2. Transformation into intermediate representations.
3. Parsing using procedural or grammar-based parsers.
4. Construction of internal semantic models.
5. Source code generation from parsed models.

A diagram illustrating the complete processing pipeline and system architecture will be included in the final version of the thesis.

In future work, the approach will be extended with AI-assisted code generation techniques based on Large Language Models (LLMs). The integration of AI methods may improve semantic interpretation of UML diagrams, automate generation of complex business logic, and support hybrid model-driven development workflows combining formal parsers with intelligent code synthesis systems.

# Conclusion

See file `zaver.tex` for information about recommended contents of the Conclusion chapter.

# Bibliography

Ortin, F., Quiroga, J., Rodriguez-Prieto, O., and Garcia, M. (2022). An empirical evaluation of lex/yacc and antlr parser generation tools. *PLOS ONE*, 17(3):e0264326.

Salunke, D., Shinde, O., Nipunge, A., Patil, D., Tekade, P., and Nikat, S. (2024). Efficient software development with uml-based code generation. In *2024 5th IEEE Global Conference for Advancement in Technology (GCAT)*, pages 1–6.

Sánchez, P., Moreira, A., Fuentes, L., Araújo, J., and Magno, J. (2010). Model-driven development for early aspects. *Information and Software Technology*, 52(3):249–273.

Xia, Y. and Glinz, M. (2003). Rigorous ebnf-based definition for a graphic modeling language. In *Proceedings of the Tenth Asia-Pacific Software Engineering Conference (APSEC'03)*, pages 186–195. IEEE.

Zhao, Z., Zhang, N., Yu, B., and Duan, Z. (2024). Generating java code pairing with chatgpt. *Theoretical Computer Science*, 1021:114879.